LESSON 03

MCP 与外部能力接入

让 AI 看见你环境之外的东西

`sqlite` `filesystem` `github` `read_query`



25 min · 一杯茶时间

AGENDA · 今日路线图

四件事，递进展开

01 MCP 是什么

给 AI 一个统一的
「呼叫外部能力」接口

类比 · 三原语 · 与 Hook 的对比

02 装上 sqlite MCP

5 分钟，零账户，
三步走完

seed_db · .mcp.json · 排错

03 让 AI 自己写 SQL

MCP 给的是接口，
不是数据

prompt 范本 · 期望对话 · 关键观察

04 反模式 + 作业

教学版全开
不等于生产配置

权限边界 · 3 反模式 · 提交清单

01

MCP 是什么

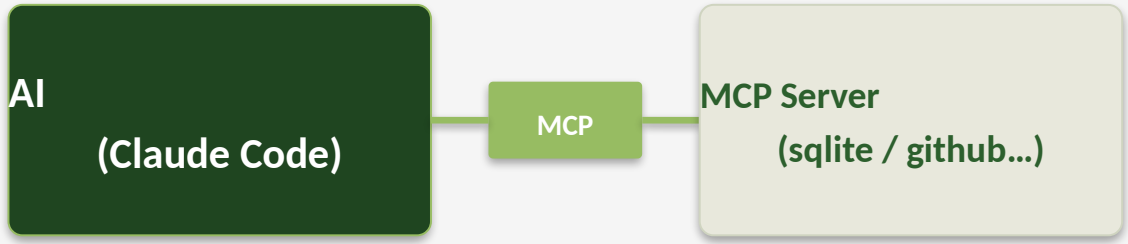
给 AI 一个统一的「呼叫外部能力」接口

MCP 之于 AI = USB 之于电脑

核心类比

USB 出现之前，每个外设都要专用驱动。
USB 出现之后，电脑只要会「说 USB」，
就能接任何键盘、U 盘、摄像头。

MCP 做的是同一件事：
只要 AI 会「说 MCP」，
就能连接任何外部能力。



1. sqlite （查询本地 DB）
2. filesystem （读写文件）
3. github （管理 Repo）

MCP 只有三种对象——但它们分工明确

Tools

AI 主动调用

AI 主动触发外部操作。
例： read_query （查 SQL ）
write_file （写文件）
create_issue （建 Issue ）

Resources

AI 主动读取

AI 像读文件一样
读取外部数据源。
例： file:///path/to/data
db://table/schema

Prompts

人 invoke

预设的 prompt 模板，
由用户（或斜杠命令）触发。
例： /summarize-pr
/weekly-report

01 · 概念定位

MCP vs Hook vs Subagent—— 各司其职

	MCP	Hook	Subagent
触发方式	AI 主动调用 Tool	事件驱动（pre/post）	AI 主动 Task 派发
上下文	主线共享	独立 shell 进程	全新独立上下文
典型用途	查询外部系统	自动化 / 日志 / 格式化	复杂子任务 / 探索
代表例子	`sqlite read_query`	bash 记录日志	审阅 / 研究

02

动手做：装上 sqlite MCP

5 分钟，零账户

cd → seed_db → .mcp.json → claude

1 准备目录

```
cd lesson3/project/
pip install requests
python seed_db.py
```

2 配置 MCP

```
cat .mcp.json
# 确认 db-path 配置正确
```

3 启动 Claude

```
cd lesson3/project &&
claude
# 必须在
lesson3/project/ 目录下
```

4 验证连接

```
问：列出 ohlc 表的字段
# AI 会自动调用 sqlite
MCP
```

准备目录 → 生成本地 prices.db

Terminal

```
$ cd ~/project
$ pip install requests

$ python seed_db.py
```

期望输出（stderr）：

```
[INFO] 正在从 Binance/MEXC 拉取 BTCUSDT 日线数据…
[INFO] 写入 prices.db ohlc 表 30 行 OK
```

为什么这两步?

requests

seed_db.py 从 Binance 拉取 K 线数据，需要此库

seed_db.py

自动建表、写入 30 天 BTCUSDT 日线 OHLC

prices.db

SQLite 文件， MCP server 只需这一个本地文件

MCP 配置文件：告诉 Claude 去哪里找能力

.mcp.json

```
{
  "mcpServers": {
    "sqlite": {
      "command": "uvx",
      "args": [
        "mcp-server-sqlite",
        "--db-path", "prices.db"
      ]
    }
  }
}
```

为什么 db-path 是相对路径?

Claude Code 在启动目录下执行，
所以 prices.db 要放在
lesson3/project/ 根目录。

用绝对路径也可以，但不便携。

关键字段说明

command: uvx （自动安装 / 运行）

args[0]: mcp-server-sqlite

--db-path: SQLite 文件路径

必须从 lesson3/project/ 目录启动

Terminal

```
$ cd lesson3/project && claude
```

为什么必须 cd 到 lesson3/project/ 目录?

1 Claude Code 以启动目录作为工作目录， MCP server 也继承这个目录

2 .mcp.json 里 db-path = "prices.db" 是相对路径，必须在同级才能找到

3 每次新开终端都要重新 cd—— 这是最常见的「明明配好了却不生效」原因

遇到问题先看这张表

现象	原因	解决方法
uvx 不存在	uv 未安装	pip install uv 或 brew install uv
不在 project 目录	Claude 找不到 prices.db	关掉 claude， cd lesson3/project/ 再重新启动
prices.db 没生成	seed_db.py 未运行或网络超时	重新运行 python seed_db.py； 检查网络或 Binance 访问
requests.HTTPError	Binance 被 GFW 屏蔽	seed_db.py 内置 binance.vision fallback， 无需代理即可访问

03

让 AI 自己写 SQL

MCP 给的是接口，不是数据

复制这条 prompt 粘贴进 claude

Prompt（整段粘贴）

```
通过 sqlite MCP 查 ohlc 表中 BTCUSDT 过去 30 天的
最高、最低、均价、最近收盘价，用一段话总结趋势。
```

注意你接下来要看什么：Claude 会自己写 SQL，通过 MCP 执行，然后用自然语言汇报结果。

03 · 期望对话

Claude 自动调用 MCP，返回真实数据

MCP(sqlite/read_query)

```
[{'max_high': 81323.52, 'min_low': 67732.01, 'avg_close': 75663.88, 'latest': 80933.55}]
```

Claude 总结

BTC 过去 30 天最高 81,323.52 USDT，最低 67,732.01 USDT，
均价 75,663.88 USDT；最近收盘 80,933.55 USDT，高于均价约 6.96%，
说明市场在经历大幅震荡（高低差约 20%）后，近期价格已从均价显著回升并接近
本周期高点，呈现出短期强势反弹态势，但距历史高点仍有一定空间，

投资者可关注能否在 81,000 USDT 上方确立支撑。

刚才发生了什么——三件值得注意的事

1

AI 自己写 SQL

你没有告诉 Claude 任何 SQL—— 它看了表结构，
自己生成了正确的聚合查询。

这正是「能力接口」的价值：你给的是意图，不是指令。

2

主线只有一次 MCP 调用

整个过程只有 read_query 被调用一次。
复杂的 SQL 构造、结果解析、汇报撰写，

全在 Claude 自身完成， MCP 只负责执行查询。

3

价值在「能力接口」而非数据

MCP 没有给 Claude 任何数据 —— 它给的是
「能查 DB 」的能力。 Claude 用这个能力

完成了端到端的数据分析任务。

两种方案，哪个更「AI Native」？

把数据塞进 Prompt

- 手动导出 CSV / SQL 结果
- 复制粘贴进对话窗口
- Token 暴增，成本升高
- 数据一变就要重新粘贴
- 无法实时查询、无法聚合

VS

MCP：给 AI「查 DB 的能力」

- Claude 自动写 SQL，实时查询
- Token 只含问答，极省
- 数据更新无需改对话
- 可聚合、过滤、关联多表
- 能力可复用到任何问题

04

权限 · 反模式 · 作业

教学版全开 ≠ 生产配置

MCP 权限分三档——按需配置

级别	配置	适用场景	注意事项
Read-only	只注册 `read_query` / `list_tables`	查询本地练习数据库（本课场景）	read_query, list_tables
Read-write	同时注册 `write_query`	让 Claude 帮你管理测试 / 开发库	read_query, write_query
生产 DB — Never	任何含真实用户数据的数据库	所有需要写操作的生产环境	权限隔离 + 审计日志 + 人工审批

学会之前先别踩坑

AP1

MCP 当 Prompt 替代品

把数百行静态文档装进 MCP Resource，让 AI 每次全量加载。

改：文档塞进 CLAUDE.md 或 system prompt；

MCP 只给「动态」数据。

AP2

接 20 个 MCP Server

一口气装 filesystem + github + slack + jira + notion + … 10 多个 server。

改：只接当前任务需要的 1-2 个；

多了会让 AI 工具选择出错。

AP3

直连生产 DB

用 mcp-server-sqlite 直连正式库，write_query 权限全开。

改：只读副本 + 明确限制 args；

写操作须经人工审批。

04 · 作业 Task 1 概览

再接一个 MCP —— 两条路选一条

A 方向 · filesystem MCP

- 在 .mcp.json 里追加 filesystem server
- args 指向 ai_lesson 仓库根目录
- 让 Claude 读 lesson1/README.md +
- 查 ohlc 表，写一段对比报告
- 提交：.mcp.json + 对话片段 + 3 句话感受

B 方向 · github MCP

- 在 .mcp.json 里追加 github server
- 配置 GITHUB_PERSONAL_ACCESS_TOKEN
- 让 Claude 拉最近 5 条 commit +
- 查 ohlc 表，写一段「活跃度 vs 行情」分析
- 提交：.mcp.json （脱敏） + 对话 + 3 句话感受

04 · 作业提交清单

30 分钟内完成，不需要写代码

✓

修改后的 .mcp.json

新增 filesystem 或 github server 配置
（可脱敏 token，但保留结构）

✓

对话会话片段截图

包含 MCP 被调用的那条消息
（能看到工具名和返回结果）

✓

3 句话感受

「接这个 MCP 最难的地方是什么？」

提交方式：截图 + .mcp.json + 3 句话感受，发到课程群即可。

「如果是生产环境你会怎么设权限？」

学完这课你能做什么

4 条 Takeaway

1 理解 MCP 架构

能解释 Tools / Resources / Prompts 三原语，
知道 MCP 与 Hook 、 Subagent 的适用边界。

2 独立接入本地 MCP

能配置 .mcp.json ， 把 sqlite / filesystem / github
等 server 接入到 Claude Code 工作区。

3 让 AI 自主查询数据

能撰写 prompt 让 Claude 自己写 SQL ，
通过 MCP 完成端到端数据分析任务。

4 识别并规避反模式

知道不要乱接 server 、不要连生产 DB 、
知道 MCP 与 Prompt 的分工边界。

下节课（Lesson 4）： 造接口——用 Skill 自定义 Claude Code 的斜杠命令

Have fun · 延伸阅读

MCP server 上哪儿找——按官方度从高到低

协议规范 `modelcontextprotocol.io`

MCP 官方 examples

`modelcontextprotocol.io/examples`

Anthropic 背书的参考实现

官方 servers 仓库

`github.com/modelcontextprotocol/servers`

filesystem / github / fetch / git

Awesome MCP Servers

`github.com/punkpeye/awesome-mcp-servers`

社区聚合，最大最杂

mcp.so

`mcp.so`

Web 注册表，可分类筛选

Smithery

`smithery.ai`

注册中心 + 一键安装 CLI

Glama

`glama.ai/mcp/servers`

带评分和使用量的注册表

Anthropic Connectors

`claude.ai → Settings → Connectors`

桌面 / 网页客户端官方目录

包来源（实操）

`npm: npx -y @modelcontextprotocol/server-*` · `PyPI: uvx mcp-server-*`

看到 server 文档里的 claude mcp add，直接抄到 .mcp.json